



UNIVERSIDADE FEDERAL DE PERNAMBUCO
CENTRO DE INFORMÁTICA
GRADUAÇÃO EM ENGENHARIA DA COMPUTAÇÃO

Marina Medeiros Lima

**Análise do Processo de Migração de Sistemas de
Arquitetura Monolítica para Microsserviço**

TRABALHO DE GRADUAÇÃO

RECIFE

2019

UNIVERSIDADE FEDERAL DE PERNAMBUCO



UNIVERSIDADE FEDERAL DE PERNAMBUCO
CENTRO DE INFORMÁTICA
GRADUAÇÃO EM ENGENHARIA DA COMPUTAÇÃO

Marina Medeiros Lima

**Análise do Processo de Migração de Sistemas de
Arquitetura Monolítica para Microsserviço**

TRABALHO DE GRADUAÇÃO

Monografia apresentada ao Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE), como requisito parcial para conclusão do curso de Engenharia da Computação, orientada pelo professor José Carlos Cavalcanti.

RECIFE

2019

Agradecimentos

Este trabalho é símbolo de uma longa jornada de graduação de uma aluna do Centro de Informática da UFPE, os anos de maiores aprendizados técnicos e pessoais já vividos, que envolveram uma miríade de pessoas. Por isso gostaria de iniciar estes agradecimentos com uma menção especial à universidade e ao CIn, em tempos onde muitos esquecem o valor e o potencial de mudança que a educação de qualidade traz, deixo aqui meu muito obrigada aos professores, servidores e auxiliares que fazem do CIn um centro de renome internacional onde pessoas incríveis tem oportunidade de aprender e construir juntas.

Gostaria de agradecer ao meu orientador, José Carlos Cavalcanti, por reforçar em mim a crença de que o espírito empreendedor está em cada um de nós, mas deve ser exercitado através da resiliência e do capricho com nossas entregas. Obrigada pela confiança e por ser meu guia na construção desse trabalho. À professora Carina Frota Alves, agradeço por ter aceitado fazer parte da minha banca examinadora, ser sua aluna em Gestão de Negócios foi um prazer e um abrir de olhos para minha formação profissional.

Agradeço à minha família pela paciência e compreensão, não apenas na construção desse trabalho, mas de toda a jornada da graduação. Sem vocês nada disso seria possível, obrigada pela atenção de longa data com minha educação e com minha saúde física e mental.

Por fim, mas de forma muito especial, agradeço ao meu noivo, Mateus Fratoni Souza, por estar ao meu lado na construção desse material e na etapa final de minha graduação. Obrigada pelo apoio e pelo incentivo à minha formação pessoal e profissional. Sua leveza e carinho trazem paz para os caminhos mais turbulentos, muito obrigada.

Resumo

A arquitetura de microsserviços é um padrão de arquitetura que desperta interesse entre engenheiros de software pela capacidade de entregas rápidas e confiáveis. Adotar essa arquitetura envolve uma transformação que engloba vários aspectos: tecnologia, metodologia, tendências organizacionais e características culturais, um escopo mais amplo do que normalmente é analisado pelas lideranças de tecnologia. Os motivos pelos quais empresas de escala global fizeram a transição do modelo monolítico para microsserviços são basicamente velocidade e segurança em tempo de produção. Velocidade na entrega de novas funcionalidades, no reparo de bugs e quebras e a capacidade de testar, adotar ou descartar novas tecnologias. Segurança pelo isolamento dos componentes, que evita propagação de falhas e quebra total do sistema.

Este trabalho traz os principais passos para realização da transição de arquiteturas monolíticas para microsserviços, baseado em extensa pesquisa bibliográfica e nos cases da Netflix, Amazon, eBay, Google, Wix e Gilt. Foi possível constatar que existem desafios claros no sistema monolítico que os microsserviços são capazes de resolver, mas que existem limitações nessa arquitetura que não fazem dela a solução definitiva para todo e qualquer nível de escala e modelo de negócio. No fim, fica claro que para cada estágio de um negócio vai existir uma arquitetura capaz de fornecer a estrutura necessária para suportar seu desenvolvimento até que seja necessário adotar um novo padrão.

Palavras-chave: Microsserviços, Monolíticos, Arquitetura de Software e Sistemas, Modelo Organizacional.

Abstract

The microservice architecture is an architectural standard that stirs interest among software engineers for the ability to deliver in a fast and reliable way. Adopting this architecture implicates in a transformation that covers several aspects: technology, methodology, organizational trends, and cultural characteristics. A broader scope than what is typically considered by technology leaders. The reasons why companies of global scale made the transition from the monolith model to microservices are basically speed and safety in production time. Speed in the delivery of new features, bug and crash repair, and the ability to test, to adopt, or to discard new technologies. Security by the isolation of the components, which avoids failure propagation with total breakdown of the system.

This work presents the main steps for the transition from monolithic architectures to microservices, based on extensive bibliographical research and on the cases of Netflix, Amazon, eBay, Google, Wix and Gilt. It was possible to verify that there are clear challenges in monolithic systems that microservices are able to solve, but there are also limitations that do not make it the definitive solution for all levels of scale and business model. In the end, it is clear that for each stage of a business, there will be an architecture capable of providing the structure needed to support its development until it becomes necessary to adopt a new architecture standard.

Keywords: Microservices, Monolith, Software Architecture, Organizational Models.

Sumário

1. Introdução	1
1.1 Contexto	1
1.2 Motivação	2
1.3 Objetivos	3
1.4 Metodologia	3
1.5 Organização	4
2. Arquitetura de Software e Sistemas	6
2.1 Particionando Sistemas	6
2.2 Arquitetura em Camadas	7
2.3 Arquitetura Baseada em Eventos	10
2.4 Arquitetura de Microkernel	13
2.5 Arquitetura de Microsserviços	14
3. Modelo Monolítico	17
3.1 Desambiguação	17
3.2 Consequências Organizacionais da Adoção de Monolíticos	18
3.3 Monolítico Como Primeiro Passo do Desenvolvimento	19
4. Arquitetura de Microsserviços	22
4.1 Arquitetura Orientada a Serviço	22
4.2 Desambiguação	24
4.4 Consequências Organizacionais da Adoção de Microsserviços	26
5. Transição da Arquitetura Monolítica para Microsserviços	30
5.1 Considerações Iniciais	30
5.2 Desafios técnicos	33
5.3 Desafios organizacionais	37
5.4 Percepções finais	39
6. Conclusão	42
7. Referências	44

Lista de Figuras

Figura 1: Representação de Arquitetura em Camadas	8
Figura 2: Representação da Comunicação na Arquitetura em Camadas	9
Figura 3: Representação da Arquitetura Baseada em Eventos, Topologia Moderador	11
Figura 4: Representação da Arquitetura Baseada em Eventos, Topologia Broker	12
Figura 5: Representação da Arquitetura de Microkernel	13
Figura 6: Representação da Arquitetura de Microserviços.....	15
Figura 7: Representação de Arquitetura Monolítica	17
Figura 8: Caminhos para Escolha da Arquitetura de Software	20
Figura 9: Representação da Arquitetura Microserviços	26
Figura 10: Representação dos Times com Arquitetura Monolítica.....	27
Figura 11: Representação dos Times com Arquitetura Microserviços	28
Figura 12: Migração de Funcionalidade Utilizando o Método da Figueira Estranguladora.....	36

Lista de Tabelas

Tabela 1: Empresas e Responsáveis Técnicos que Realizaram a Transição de Sistemas Monolíticos para Microserviços.....30

Tabela 2: Sumário de Passo a Passo para Transição para Microserviços.....40

1. Introdução

De acordo com Nadareishvili et al. [\(2016\)](#) a arquitetura de microsserviços tem emergido como um padrão que desperta interesse entre engenheiros de software pela capacidade de entregas rápidas e confiáveis. Adotar essa arquitetura envolve uma transformação que engloba vários aspectos: tecnologia, metodologia e tendências organizacionais. Por isso, as empresas que buscam colher os benefícios da arquitetura de microsserviço precisam considerar um escopo mais amplo do que apenas tecnologia.

O objetivo desse trabalho de graduação é contribuir para um maior entendimento do porquê empresas estão adotando a arquitetura de microsserviços, quais as motivações e desafios e, principalmente, de que forma empresas que adotam o modelo de arquitetura monolítico até hoje podem se beneficiar da transição para microsserviços.

1.1 Contexto

Para Conway [\(1968\)](#), a estrutura de comunicação das empresas é um elemento limitante para o projeto de soluções. Se não for possível para a empresa repensar sua estrutura de comunicação, esta será o gargalo para o desenvolvimento de inovação. Isso também implica que para criar algo novo é preciso estar aberto a transformações que vão impactar toda a organização.

Paralelamente, para Foster [\(1986\)](#), existe uma briga constante no mercado entre empresas inovadoras (*attackers*) e empresas que querem manter o status quo (*defenders*). Empresas inovadoras não têm medo de mudar sua solução, seu produto ou sua estrutura organizacional, pois elas entendem que esse é um pequeno preço a se pagar para obter vantagem competitiva no longo prazo. Alguns exemplos práticos dessa visão de Foster serão abordados nesse trabalho, como Amazon, Netflix, eBay e Google; empresas que materializam este conceito: *“It is about the inexorable and yet stealthy challenge of new technology and the economics of sub situation which force companies to behave like the mythical phoenix, a bird that periodically crashed to earth in order to rejuvenate itself”*.

Numa publicação mais recente, Foster [\(2001\)](#) traz a importância da destruição criativa como forma das empresas aumentarem *market share*. Em outras palavras, o mercado não está

preocupado com os desafios do time de engenharia em entregar novas soluções; por isso, da porta para dentro, as empresas precisam se valer de práticas que permitam que elas se comportem como *attackers* ([Foster, 1986](#)). Para um líder de tecnologia (CEO, CTO, VP de Engenharia) isso significa garantir que as soluções sejam entregues da forma mais positiva possível: rápida, sem bugs, sem *delays* de atualização, com novas *features*, mas que elas não causem quebra; com novos clientes entrando na base, mas que o sistema não caia; e com uma usabilidade cada vez melhor. E esse time precisa ter liberdade para mudar não apenas a arquitetura de software do sistema, mas também a forma como todo o time interage entre si e com o restante da organização ([Conway, 1968](#)).

Esse trabalho de graduação traz o estudo da arquitetura de microsserviços como alternativa à arquiteturas monolíticas para alcançar velocidade e confiabilidade de entrega em serviços de software para empresas que almejam escalar. A Netflix foi um dos primeiros cases de sucesso de migração em produção da arquitetura monolítica para a de microsserviço, trazendo resultados e boas práticas em 2009; o case da Amazon é de 2010. Vale ressaltar aqui que a Amazon não utiliza o termo “microsserviço” e sim “arquitetura baseada em células” e “evolução natural do SOA”, a caracterização de cada uma delas será feita adiante. O eBay, logo após a Amazon, com a transição em 2011 e o Google, mais tarde, em 2013.

1.2 Motivação

A arquitetura de software ajuda no desenvolvimento de sistemas ao permitir uma visualização dos componentes necessários e a forma como eles irão interagir. Definir uma arquitetura antes de iniciar o desenvolvimento de um sistema afeta toda a vida do mesmo em termos de manutenção, expansão e a capacidade de testar ([Richardson, 2018](#)).

A arquitetura tratada na literatura como “monolítica” é a mais fácil para começar o desenvolvimento de um software pela rapidez em projetar, implementar e colocar em produção. No entanto, conforme novas funcionalidades são adicionadas, as tecnologias vão se atualizando e novas demandas vão surgindo, o monolítico tende a ficar cada vez mais difícil de ser otimizado: adicionar funcionalidades se torna uma atividade demorada e custosa pelos bugs que aparecem em partes do sistema que não deveriam ser atingidas, fazer o *deploy* se

torna um desafio pelo tamanho do sistema, *memory leak* em uma funcionalidade compromete o resultado de outra completamente diferente, e o sistema quebra (sistemas locais) ou sai do ar (sistemas web).

Empresas como Netflix, Amazon, eBay, Google, Wix e Gilt são alguns exemplos de aplicações web que migraram da arquitetura monolítica para a arquitetura de microsserviços em plena operação do serviço, e trouxeram para o mercado novas formas de pensar em design organizacional. O desafio enfrentado por muitas empresas hoje é entender se essa migração faz sentido para seu modelo de negócio, e, se sim, qual o melhor momento para mudar a arquitetura e quais as implicações dessa decisão.

1.3 Objetivos

Esse trabalho de graduação tem como propósito mais amplo fazer uma revisão bibliográfica dos conceitos de arquitetura de software buscando entender as motivações de grandes empresas de tecnologia estarem migrando para a arquitetura de microsserviços.

De maneira geral, os objetivos do trabalho são:

- Investigar quais são os indícios de que um serviço digital se beneficia mais de uma arquitetura de microsserviço, e quais são as implicações envolvidas nessa decisão;
- Apresentar o passo a passo da transição da arquitetura monolítica para microsserviço baseado em cases.

De forma específica o trabalho traz:

- Os conceitos de padrões de arquitetura de software e sistemas, o entendimento do modelo monolítico e microsserviços, os prós e contras de cada um, os principais aprendizados que grandes empresas de tecnologia reuniram em suas transições e a análise da transformação que a transição causa não apenas na área de tecnologia das empresas, mas em toda organização.

1.4 Metodologia

A pesquisa realizada para cumprir os objetivos expostos anteriormente se baseou em cases de transição de microsserviços divulgados por grandes empresas que oferecem serviços

de tecnologia via web em nível global: Netflix, Amazon, eBay, Google, Wix e Gilt. As informações vêm de blogs dos CTOs e VPs de engenharia responsáveis pela transição, com conteúdo escrito e visual obtidos online.

Para melhor apresentar os conceitos propostos pelas empresas, foi feita uma extensiva pesquisa bibliográfica para os temas de padrões de arquitetura de software e sistemas, modelos monolíticos, arquitetura de software orientada à serviços, arquitetura de software de microserviços e APIs para definição de termos, estrutura, prós e contras de cada arquitetura.

1.5 Organização

A divisão do trabalho está feita de forma a elucidar os principais conceitos de arquitetura de software, modelo monolítico e arquitetura de microserviços, para entender as consequências da utilização de cada um e então entrar no processo de transição.

O processo de transição da arquitetura monolítica para microserviços foi abordado de forma a trazer os principais aprendizados que as empresas que já realizaram a transição compartilharam com a comunidade.

O documento está dividido em 5 capítulos:

- **Capítulo 1, Introdução:** contextualiza o trabalho em termos do status quo da temática, explicando a motivação para o trabalho, os objetivos definidos, a metodologia utilizada e a estrutura do documento.
- **Capítulo 2, Padrões de Arquitetura de Sistemas:** é apresentado o contexto de arquitetura de software com definição e histórico, em seguida os principais aspectos (positivos e negativos) da arquitetura em camadas, da arquitetura baseada em eventos, da arquitetura de microkernel e da arquitetura de microserviços.
- **Capítulo 3, Arquitetura Monolítica:** por não se tratar de um padrão de arquitetura, o termo monolítico é explicado a partir das características que fazem um sistema ser considerado um monolítico, as implicações organizacionais de se utilizar essa arquitetura e a visão de alguns autores sobre qual é a melhor arquitetura para iniciar o desenvolvimento de um sistema.

- **Capítulo 4, Arquitetura de Microserviços:** o histórico da arquitetura é apresentado, seguido de uma comparação com a arquitetura orientada a serviços, em seguida os pontos positivos e negativos de utilizar a arquitetura de microserviço e por fim a estrutura organizacional que comporta esse padrão.
- **Capítulo 5, Transição da Arquitetura Monolítica para Microserviços:** os principais cases de transição estão organizados nesse capítulo inicialmente com os problemas motivadores para a transição, os desafios técnicos superados e não superados e os desafios organizacionais enfrentados pelas empresas.
- **Capítulo 6, Considerações Finais:** aqui é trazido a conclusão do trabalho e sugestões de trabalhos futuros.

2. Arquitetura de Software e Sistemas

Este capítulo traz os principais conceitos de arquitetura de software: porque ela é usada e de que forma é possível se beneficiar da modularização que ela traz. Em seguida alguns padrões de arquitetura comumente utilizados são trazidos e analisados do ponto de vista de componentes e comunicação.

2.1 Particionando Sistemas

Arquitetura de software e sistemas é um conceito nascido nos anos 60, com o intuito de facilitar o desenvolvimento de software, que ganhava cada vez mais espaço (e mais complexidade) nas empresas. Os sistemas de software são criados para satisfazer objetivos de negócio, já a arquitetura software e sistemas é a ferramenta que facilita a constatação de que o sistema atingiu os objetivos. *“While the path from abstract goals to concrete systems can be complex, the good news is that software architectures can be designed, analyzed, documented, and implemented using known techniques that will support the achievement of these business and mission goals. The complexity can be tamed, made tractable.”* ([Bass, Clements e Kazman, 2013](#)).

Para Shoup ([2014](#)), não existe arquitetura perfeita para todos os produtos em todas as escalas, um conceito presente também em Fowler ([2005](#)) e Nadareishvili et al ([2016](#)). Fowler cunhou o termo “arquiteturas de sacrifício” para aquelas arquiteturas que são adotadas em alguns períodos de existência do sistema para serem descartadas em breve. Shoup aprofunda trazendo que arquiteturas são temporárias e que a comunidade técnica precisa estar de acordo com isso, pois para ele “é mais um privilégio do que um fardo poder reescrever um sistema que foi superado.” ([Shoup apud Fowler, 2014](#)).

Essa preocupação em organizar a forma de criar software e definir uma lógica antes de começar a escrever código veio de Edsger Dijkstra ([1968 apud Bass, Clements e Kazman, 2013](#)) quando ele trouxe a ideia de criar um sistema operacional separando-o em camadas de forma a limitar a comunicação dos programas para que só se relacionassem com programas de camadas adjacentes. Seu objetivo era garantir a integridade do sistema desenvolvido e

facilitar sua manutenção. Já em 1970, David Parnas, contribuiu com a criação de alguns padrões comuns a todas as arquiteturas como quebrar o sistema em módulos focando em aumentar a capacidade de reuso e manutenção, ter uma forma clara de comunicação entre esses módulos, o princípio de detecção e correção de erros encadeados (que deu origem às *exceptions*), e o reconhecimento de que a estrutura adotada pode influenciar na qualidade do sistema final ([Parnas apud Bass, Clements e Kazman, 2013](#)).

Assim como a arquitetura no âmbito da construção civil, a arquitetura de software não está preocupada em “como” cada elemento irá performar suas tarefas (esse é o trabalho do engenheiro), o objetivo é garantir a existência de elementos que entreguem resultados ao usuário. Por outro lado, é importante ressaltar que, diferentemente da arquitetura na construção civil, na arquitetura de software o usuário tem zero visibilidade da arquitetura por trás do sistema desenvolvido.

Dessa forma, a arquitetura de software e sistemas existe para definir o escopo de desenvolvimento, suas propriedades e como ele irá se relacionar com outros sistemas. Além disso, como Dijkstra pontuou, é mais fácil entender um sistema quando visto por partes e alocar responsáveis por cada parte facilita manutenção. A forma como o sistema será quebrado é o que vai nomear a arquitetura, e a seguir, algumas dessas formas são analisadas.

2.2 Arquitetura em Camadas

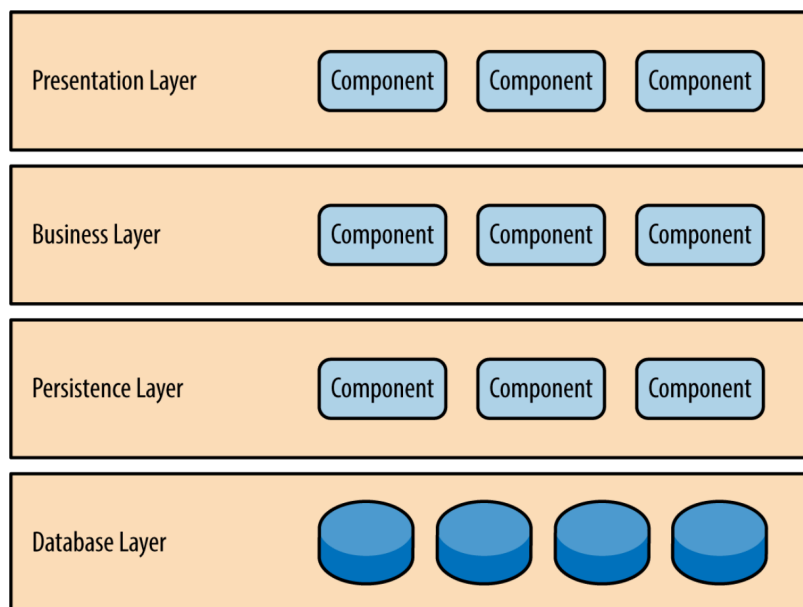
Começando pela arquitetura em camadas, a mais próxima da concepção inicial de Dijkstra para modularização mencionada no item 2.1, este padrão é amplamente difundido entre arquitetos, designers e desenvolvedores. A arquitetura em camadas é um padrão de propósito geral que funciona como uma boa alternativa para começar o desenvolvimento de aplicações, em especial quando não se sabe exatamente qual será a entrega final da aplicação ([Richards, 2015](#), [Nadareishvili, 2015](#) e [Richardson, 2015](#)).

Sua principal característica é a definição de camadas com papéis bem definidos em uma estrutura hierarquizada, normalmente do componente de maior abstração para o de menor abstração. Não existe uma regra para o número de camadas e seus papéis, mas a maioria das empresas se baseia em: apresentação (maior abstração, em geral a interface do

usuário), negócio, persistência e banco de dados (menor abstração), como pode ser visto na figura 1. Em geral, pequenas aplicações tem em torno de três camadas e grandes aplicações mais de cinco ([Richards, 2015](#)).

Um dos grandes diferenciais da arquitetura em camadas está nessa separação de atribuições de acordo com a função de cada camada. Essas atribuições são claras e rígidas no sentido de que os componentes interagem em nível lógico apenas com outros componentes da mesma camada. Isso ajuda a construção de componentes com papéis e responsabilidades claras, além de deixar o desenvolvimento mais objetivo, fácil de testar, gerir e manter o sistema.

Figura 1: Representação de Arquitetura em Camadas



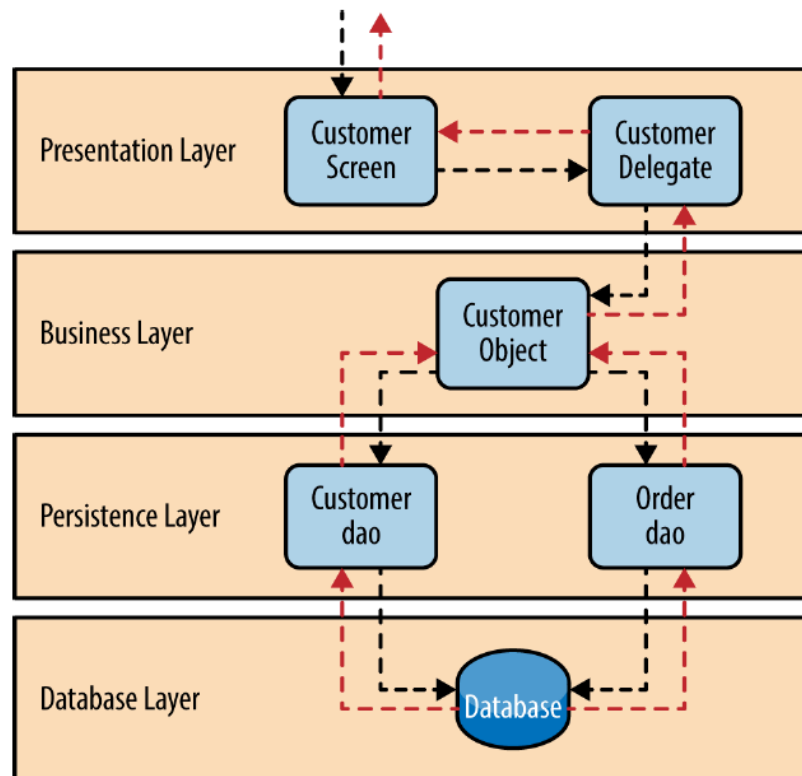
Fonte: *Software Architecture Patterns*, Mark Richards ([2015](#))

Essa definição de atribuições e papéis impacta diretamente na construção dos times de desenvolvimento. Essa relação da estrutura organizacional e o projeto de software é explorada na Lei de Conway ([1968](#)): “organizações que projetam sistemas (no conceito mais amplo) estão limitadas a construir projetos que são cópias da estrutura de comunicação dessas organizações”. Richards ([2015](#)) exemplifica: “O padrão de arquitetura em camadas é muito parecido com a estrutura organizacional e de comunicação encontrada na maioria das empresas, fazendo dela a escolha mais natural para a maioria dos desenvolvedores de aplicações”.

Uma regra herdada de Dijkstra é a comunicação entre camadas: só é possível acessar aquelas que são adjacentes e existe uma definição clara de interfaces e escopo de cada componente. Estes componentes tendem a se tornar fortemente acoplados (trocam informações no nível de lógica do negócio), o que dificulta sua manutenção, testes e escala. A comunicação entre componentes pode ser vista na figura 2.

Aplicações que seguem essa arquitetura enfrentam alguns desafios. Conforme vai crescendo, a aplicação se torna mais difícil de responder à mudanças de forma ágil. Mesmo sendo capaz de limitar mudanças à camadas específicas, é custoso fazer alterações devido à natureza monolítica da maioria das implementações com componentes fortemente acoplados ([Richards, 2015](#)).

Figura 2: Representação da Comunicação na Arquitetura em Camadas



Fonte: *Software Architecture Patterns*, Mark Richards ([2015](#))

Em resumo, a arquitetura em camadas possui pontos positivos como facilidade de desenvolvimento e de testes (tanto do sistema como um todo, como de componentes ou camadas específicas). Por outro lado, conforme vai crescendo, a aplicação que utiliza esse

padrão de arquitetura não é o ideal para quem busca alta performance, agilidade em desenvolvimento e escalabilidade.

2.3 Arquitetura Baseada em Eventos

A arquitetura baseada em eventos é um padrão de arquitetura distribuída assíncrona, comumente utilizada por aplicações escaláveis, mas se trata de um padrão adaptável que pode ser usado para aplicações pequenas e simples ou grandes e complexas.

Algumas definições cabem para um melhor entendimento do padrão: o sistema a ser distribuído significa que todos os componentes da arquitetura são desacoplados e acessados através de algum protocolo de acesso remoto, e a comunicação assíncrona significa que a transmissão de dados pode ser feita intermitentemente em um fluxo estável e até paralelo. No contexto da arquitetura, um evento é qualquer mudança significativa de estado. Nesse padrão, componentes de processamento de evento são desacoplados e dedicados (resolvem apenas um propósito específico) que recebem e processam eventos.

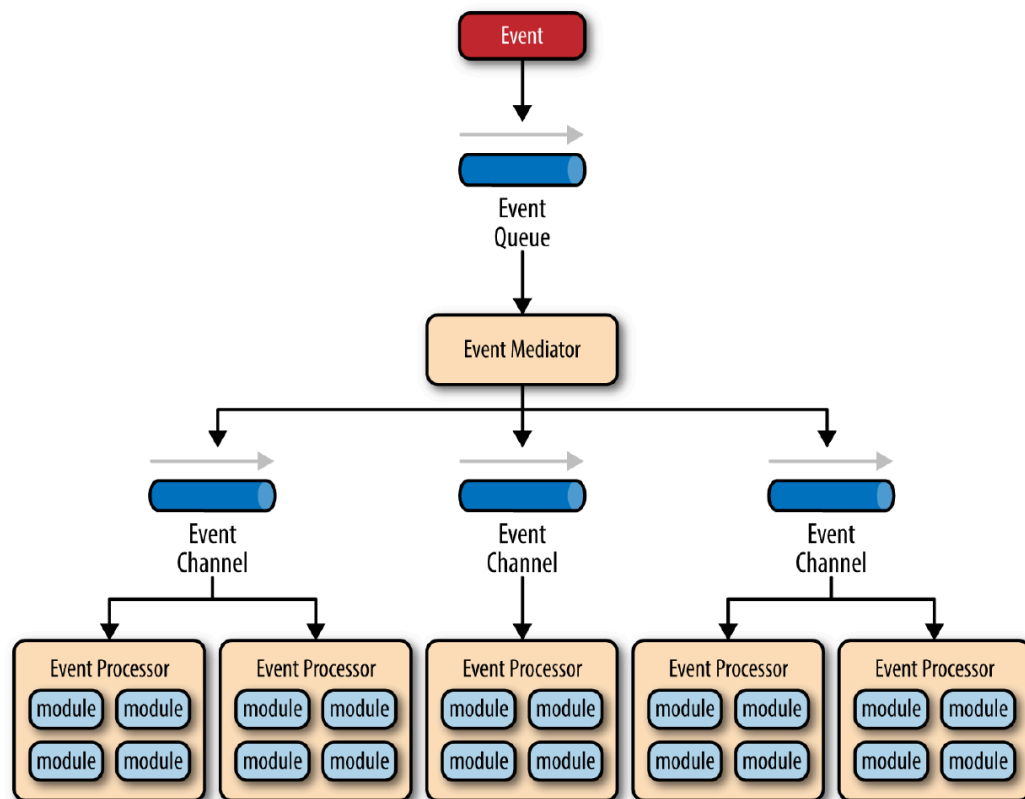
A arquitetura baseada em eventos disponibiliza duas topologias possíveis: *mediador* e *broker* e eles se diferenciam na maneira como os eventos são orquestrados no sistema. Na estrutura de *mediador*, os eventos serão arbitrados por uma entidade mediadora antes de desencadear outros eventos. Já na topologia *broker*, não existe um mediador: os eventos são autônomos em suas decisões de desencadeamento.

O modelo *mediador* é útil para instruções quebrados em várias etapas, e que necessitem de um árbitro para intermediar o processamento dos eventos que precisam ser feitos em ordem, ou que podem acontecer paralelamente ([Richards, 2015](#)). Os componentes dessa topologia são: o mediador de eventos (*event mediator*), as filas de eventos (*events queue*), os canais de eventos (*event channels*) e os processadores de eventos (*event processors*). Esses componentes podem ser visualizados da figura 3.

Para exemplificar, em um sistema de uma seguradora de imóveis que utilize essa topologia, se um cliente vai se mudar e precisa alterar seu endereço de cadastro, o evento é enviado à fila de eventos que organiza o acesso ao mediador de eventos. O mediador recebe o pedido e faz a orquestração com os canais que precisarão ser acionados para realizar essa

alteração enviando eventos assíncronos aos canais de eventos para executar os processos necessários (nesse exemplo pode-se ressaltar: alterar endereço de cadastro, processo de cotação e o processo de solicitações). Os processadores de eventos, que acompanham os canais de eventos, recebem o evento do mediador do evento e executam uma lógica de negócios específica para processar o evento. É comum ter entre dezenas e centenas de filas de eventos em uma arquitetura orientada a eventos ([Richards, 2015](#)).

Figura 3: Representação da Arquitetura Baseada em Eventos, Topologia Moderador



Fonte: *Software Architecture Patterns*, Mark Richards ([2015](#))

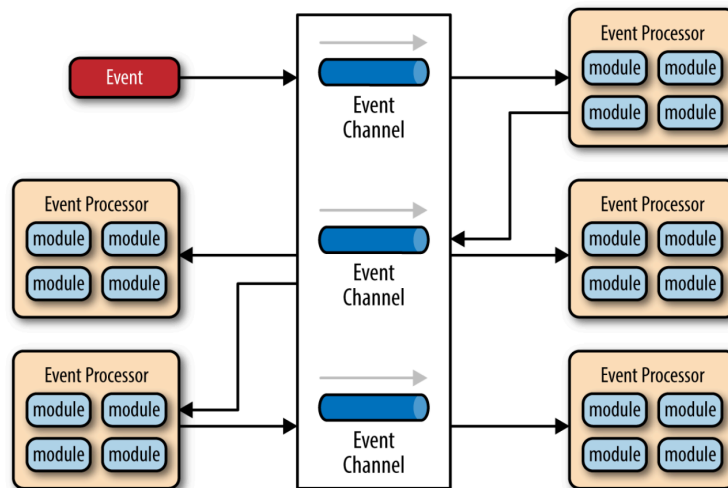
É importante salientar que o mediador de eventos não executa nenhuma lógica de negócio com o evento recebido, apenas os processadores de evento fazem isso. O mediador é responsável apenas por enviar o evento aos canais apropriados para que a ordem seja executada de forma satisfatória.

O modelo *broker* funciona de forma diferente do modelo mediador pela inexistência de uma entidade responsável pela orquestração dos eventos. Neste, os eventos se comunicam

através de um fluxo de mensagens através dos processadores de eventos como mostrado na figura 4.

Usando o mesmo exemplo do sistema de seguradora de imóveis, se um cliente vai se mudar, o evento de alterar endereço será responsável por desencadear todas as mudanças no sistema. Como não existe um mediador, o evento vai diretamente ao canal de eventos e o processador de evento recebe o comando de alteração de endereço. Ao realizar a tarefa, este envia um evento de volta ao canal informando que a alteração foi feita. Quando esse evento é disponibilizado no canal, os processadores da cotação e de solicitação acompanham e podem se atualizar e disparar novos eventos de volta ao canal para que outros processadores façam alterações necessárias até que todos os processadores pertinentes estejam atualizados e não reste mais nenhum evento no canal ([Richards, 2015](#)).

Figura 4: Representação da Arquitetura Baseada em Eventos, Topologia Broker



Fonte: *Software Architecture Patterns*, Mark Richards ([2015](#))

A arquitetura baseada em eventos não é um modelo trivial de adotar devido à sua natureza distribuída e assíncrona. O desenvolvimento apresenta desafios, pois o tratamento de erros (e seus desencadeamentos em outros processadores de eventos) requer ferramentas mais sofisticadas. Também é mais complexo de realizar testar tanto componentes isolados quanto um fluxo completo de eventos. ([Richards, 2015](#)).

Aqueles que adotam a arquitetura baseada em eventos o fazem pelo alto nível de performance e capacidade de escala. Se implementada respeitando o padrão também será fácil de fazer *deploy*, já que os elementos são desacoplados e independentes, o que também permite

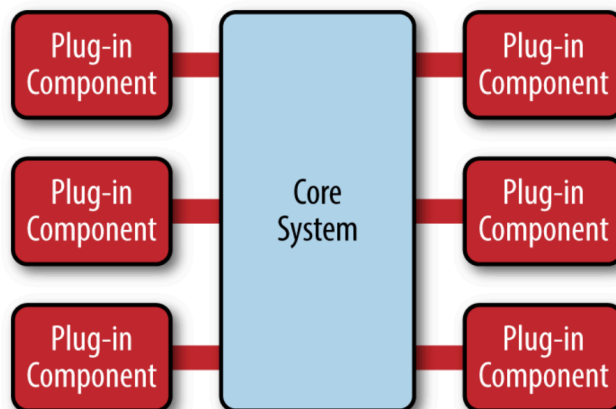
que o sistema seja mais facilmente incrementado. É válido ressaltar que os componentes do processador de eventos descritos dessa arquitetura podem ser implementados usando o padrão de arquitetura de microsserviços.

2.4 Arquitetura de Microkernel

O padrão de arquitetura de microkernel é comumente utilizado para sistemas empacotados e disponibilizados em versões para download como um produto fechado (*product-base applications*) e aceita que aplicações *plug-ins* (módulos plugáveis) interajam com o sistema central. Estes são os principais componentes do padrão: um sistema central e *plug-ins*.

Os *plug-ins* são módulos independentes e adicionais que provêm extensibilidade, flexibilidade e são isolados dos recursos do sistema central. Esse, geralmente possui o mínimo de funcionalidades necessárias para fazer o sistema ser operacional (não por coincidência, sistemas operacionais adotam arquitetura de microkernel e por isso recebe esse nome). Do ponto de vista de negócio, o sistema central possui toda a lógica, enquanto os *plug-ins* trazem casos e regras especiais. É possível visualizar os componentes da arquitetura na figura 5.

Figura 5: Representação da Arquitetura de Microkernel



Fonte: *Software Architecture Patterns*, Mark Richards ([2015](#))

Essa arquitetura tem uma peculiaridade interessante que é sua capacidade de se acoplar a outras arquiteturas, visto o exemplo dos sistemas operacionais que suportam a execução de diversos tipos de aplicações. O oposto também é possível e um sistema de

arquitetura em camadas pode preferir resolver um problema específico com uma aplicação microkernel de forma isolada.

A arquitetura de microkernel deve ser a primeira escolha para *product-based applications*, principalmente quando se planeja trazer recursos adicionais que poderão ser acessados separadamente (ou vendidos). Também é uma característica do padrão que o sistema central se estabilize de forma robusta e não requeira muitas alterações, por isso apesar de não ser recomendada sempre, essa arquitetura se encaixa bem para produtos de prateleira.

Pela sua natureza de ter um sistema central, é preciso projeto cuidadoso e governança sobre as alterações e versionamento para realizar alterações, o que deixa sua implementação complexa. Normalmente, as soluções que adotam microkernel são pequenas e não provêm muita escala, além do que for possível de estender através dos *plug-ins*.

2.5 Arquitetura de Microserviços

O padrão de arquitetura de microserviços vem ganhando espaço no mercado como substituto para aplicações monolíticas e arquiteturas orientadas a serviços, que serão explorados a fundo nos capítulos 3.1 e 4.1, respectivamente. Assim como a arquitetura baseada em eventos, microserviços também é um padrão distribuído.

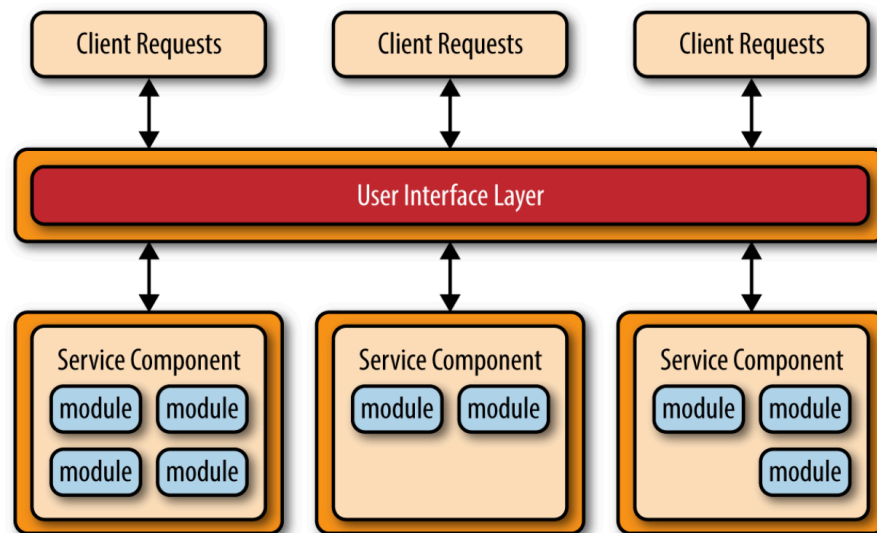
Primeiramente, faz sentido pensar em serviços aqui como componentes de serviços: eles podem ser uma unidade única ou um conjunto de unidades, que resolvem um problema específico. “Projetar o nível certo de granularidade do componente de serviço é um dos maiores desafios em uma arquitetura de microserviços”, Richards (2015). A arquitetura consiste de componentes de serviços com módulos internos e objetivo de negócio claro e independente. Na figura 6 é possível ver uma representação da arquitetura.

Existem três topologias comumente utilizadas na arquitetura de microserviços: baseada em API REST, em aplicação REST e em mensagens centralizadas. Essas topologias definem a forma como as solicitações dos clientes são tratadas e o tamanho dos componentes de serviços.

Uma API (*Application Programming Interface*, ou interface de programação de aplicação) é um conjunto de rotinas, ferramentas e protocolos para criação de uma aplicação e

independente de plataforma. ([Rito apud Daniel Jacobson, Greg Brail e Dan Woods](#)). A topologia baseada em API REST (*Representational State Transfer*, ou Transferência de Estado Representacional) é útil para sites que disponibilizem serviços através de uma API. Os serviços são acessados usando interfaces REST implementadas pela API e os componentes de serviço possuem um ou dois módulos internos (são pequenos e objetivos).

Figura 6: Representação da Arquitetura de Microsserviços



Fonte: *Software Architecture Patterns*, Mark Richards ([2015](#))

Já a topologia baseada em aplicação REST trata a comunicação de maneira tradicional, sem usar uma API. A camada de interface do usuário é implementada com *Web Service*, que acessa remotamente componentes de serviço independentes por meio de interfaces baseadas em REST.

A diferença entre a topologia API REST e aplicação REST está no tamanho dos componentes de serviços, que no último tendem a ser maiores, mais granulares e resolvem parte de um objetivo do negócio. Essa topologia acaba sendo mais comum para pequenos sistemas de baixa complexidade por ser de difícil manutenção.

Por fim, a topologia baseada em centralização de mensagens adiciona uma camada na topologia de aplicação REST com um *broker* de mensagens. Essa camada não tem grande poder de orquestração, transformação ou roteamento complexo, apenas permite acesso a componentes de serviços remotos.

Dentre as topologias apresentadas, a API REST é a que melhor colhe os benefícios da arquitetura de microsserviços, pois consegue manter a facilidade de desenvolvimento, respondendo às mudanças de forma ágil com capacidade de *deploy* dos componentes de forma independente permitindo que a aplicação escale de forma salutar.

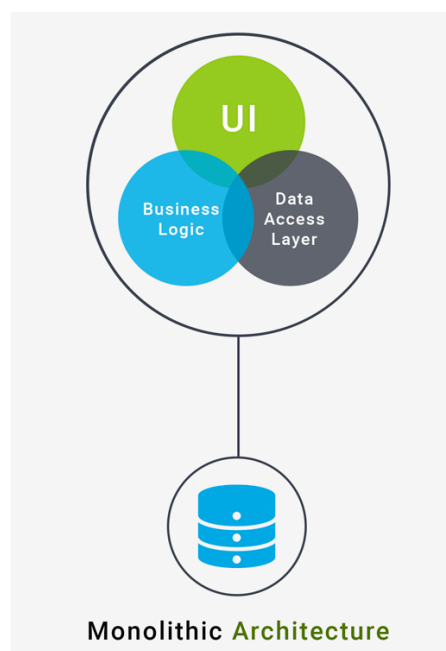
3. Modelo Monolítico

Por não se tratar de um padrão de arquitetura e sim de uma característica presente em vários padrões, este capítulo se destina a caracterizar o que faz de um sistema, um monolítico. Também são levantados os principais aspectos da construção de um software nessa arquitetura, a estrutura organizacional de uma empresa que a aplica, e finalmente, encerra com o porquê de alguns especialistas acreditarem que o melhor caminho de desenvolvimento está em construir um sistema monolítico para posteriormente quebrá-lo em microsserviços.

3.1 Desambiguação

O termo “monolítico” é utilizado quando o sistema é construído como uma única unidade: todos seus componentes estão juntos em uma única plataforma, por exemplo um único cliente com sua UI (*User Interface*, interface do usuário), suas regras de negócio e sua camada de acesso à dados, e arquivo Java WAR. Na Figura 7 é possível ver a arquitetura de um sistema web do lado do apenas o banco de dados de forma independente ao sistema ([Awasthi, 2017](#)).

Figura 7: Representação de Arquitetura Monolítica



Fonte: *Microservices vs Monolithic Architecture*, Mayank Awasthi ([2017](#))

Essa estrutura básica possui alguns benefícios ([Richardson, 2018](#), em tradução livre):

- Simplicidade para desenvolver: as ferramentas disponíveis para desenvolvedores geralmente focam em construir uma aplicação única;
- É fácil de fazer mudanças radicais na aplicação: mudar o código e o esquema do banco de dados, o *build* e o *deploy*;
- É direta para testar: os desenvolvedores escrevem testes fim a fim que simplesmente iniciam a aplicação, enviam requisições e analisam o resultado de forma direta;
- Fazer o *deploy* é simples: basta subir o arquivo único (ex: um arquivo *Java WAR*) no servidor obedecendo as ordens de dependência;
- É fácil de escalar: basta replicar as instâncias da aplicação de forma controlada por um balanceador de carga.

Construir uma aplicação de arquitetura monolítica é simples e geralmente lida apenas com uma única linguagem de programação, além de poder ser feito por apenas um desenvolvedor rapidamente. Se o objetivo da aplicação é testar uma nova ferramenta de desenvolvimento ou IDE (*Integrated Development Environment*, ou em português Ambiente de Desenvolvimento Integrado), a aplicação monolítica é a forma mais rápida para alcançar os resultados. Quando o objetivo da aplicação é comercial, é preciso olhar com mais atenção aos desafios desse modelo.

3.2 Consequências Organizacionais da Adoção de Monolíticos

Conforme a solução vai sendo utilizada no mercado e o que era apenas um software vira a solução de uma empresa, é preciso olhar para as estruturas que esta empresa pode assumir. “*An organizational structure defines how job tasks are formally divided, grouped, and coordinated*” ([Robbins, 2013](#)). Desde a revolução industrial, o mercado tem adotado estruturas com fortes padrões hierárquicos e alto nível de especialização por área e que geralmente envolve os seguintes componentes:

- “Os funcionários geralmente trabalham em departamentos ou equipes com escopo e responsabilidades fixas.’
- ‘Os funcionários têm um gerente que definem as tarefas a serem feitas.’
- ‘Os funcionários têm cargos claros e salários predeterminados para este cargo.’
- ‘Os funcionários recebem o suficiente para comprar produtos e participar de forma significativa na economia’
- ‘Os funcionários estão sujeitos a uma hierarquia de gestão burocrática que é responsável pela tomada de decisão.’
- ‘Funcionários trabalham em um escritório com seus colegas.” ([Hensal, 2017](#) apud [Clark, 1980](#))

Revisitando a Lei de Conway ([1968](#)) citada no capítulo 2.1, essa estrutura faz sentido para aplicações monolíticas (e inclusive possui características que lembram a arquitetura em camadas): os times de desenvolvimento são separados por expertise e eles só se comunicam entre si através da liderança, que também define quais serão os próximos passos do desenvolvimento; esses passos consistem em entender se existem bugs, quais serão corrigidos primeiros e quais novas funcionalidades serão adicionadas. Por sua vez, essa demanda virá da área de suporte que possui seu próprio líder e será responsável por repassar os bugs e problemas reportados pelos clientes.

No eBay, por exemplo, existia um comitê de aprovação de arquitetura que definia o que seria modificado no sistema e era responsável por encaixar todas as alterações nas camadas (e conseqüentemente nos times) que fizessem mais sentido. Segundo Randy Shoup ([2015](#)), a lentidão gerada pelo comitê era tamanha que algumas funcionalidades perdiam sentido de serem aprovadas e implementadas depois de algumas semanas.

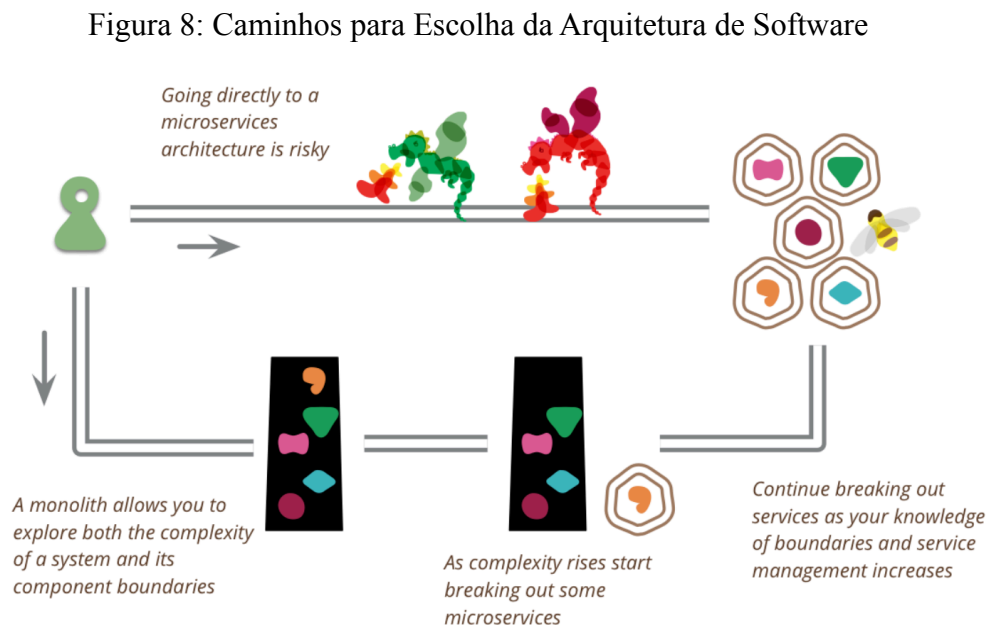
3.3 Monolítico Como Primeiro Passo do Desenvolvimento

Assim como já foi pontuado por Richards ([2015](#)) no capítulo 2.2, no geral, o padrão de arquitetura em camadas (o monolítico mais tradicional) é a forma mais simples de iniciar o desenvolvimento de aplicações especialmente quando não se sabe ainda quais as grandes entregas em termos de negócio daquele sistema. É fato que a arquitetura de microsserviços

consegue resolver os problemas mais comuns encontrados em arquiteturas monolíticas e arquiteturas orientadas a serviço (que serão tratadas com mais detalhes no capítulo 4) como robustez, capacidade de escala e entregas contínuas. Porém, esses desafios surgem conforme o sistema vai ganhando tração de uso e robustez.

Para Fowler (2015), mesmo quando a aplicação tem o objetivo de crescer (em número de usuários ou de funcionalidades), deve-se começar o desenvolvimento de uma aplicação monolítica. Segundo ele, a complexidade da criação de um sistema distribuído é grande demais para uma solução que ainda precisa ser validada. *“Microservices are a useful architecture, but even their advocates say that using them incurs a significant Microservice Premium, which means they are only useful with more complex systems”*. Ele chamou de “prêmio do microserviço” o ônus da manutenção dos vários serviços envolvidos na aplicação, mesmo que inicial. Essa complexidade, de criação e manutenção de vários serviços, não compensa o risco da aplicação não ser comercialmente aceita. Desenvolver um sistema, levar à mercado e testar sua solução com o usuário será mais rápido utilizando uma arquitetura monolítica.

A figura 8 ilustra o caminho sugerido por Fowler para a criação de um sistema, inicialmente monolítico, que cresce, aumenta a complexidade e é quebrado em microserviços.



Fonte: *Monolith First*, Martin Fowler (2015)

Para Nadareishvili et al. (2016) sistemas de software tendem a ficar mais complexos conforme escalam em termos de escopo, volume e interações com usuários. O papel do arquiteto de software está em encontrar o equilíbrio entre velocidade e segurança: velocidade de entrega de novas funcionalidades, de reparação de quebras e de performance; e segurança na utilização, nas novas funcionalidades e no armazenamento de dados.

Além disso, Nadareishvili et al. (2016) também traz uma visão complementar à Fowler, considerando que “apenas pela criação e *ownership* de aplicações monolíticas, os limites dos serviços certos serão identificados”. Ou seja, apenas com o sistema funcionando por completo será possível quebrá-lo em serviços. Ele também argumenta que a transição da arquitetura monolítica para microserviço segue o princípio da Lei de Gall que diz que “todo sistema complexo é derivado de um sistema simples que funcionava”. Por fim, encerra seu argumento com o fato de que as primeiras empresas a utilizarem a arquitetura de microserviços com sucesso fizeram a transição depois de alcançarem uma barreira para escalar seus negócios de maneira saudável com a antiga arquitetura.

4. Arquitetura de Microsserviços

Esse capítulo começa com a explanação da arquitetura orientada a serviço, uma das principais fontes de inspiração para a arquitetura de microsserviços. Em seguida, é feita uma desambiguação do termo e os principais aspectos da arquitetura são levantados. Por fim, uma descrição do impacto organizacional traz as principais consequências em termos de gente e gestão para as empresas.

4.1 Arquitetura Orientada a Serviço

“*Service Oriented Architectures, Part I*” foi um dos primeiros registros de pesquisa no tema de Arquiteturas Orientadas a Serviços (SOA, *Service Oriented Architecture*), publicado pelo Gartner Group em 1996 ([Schulte e Natis](#)). Segundo o material, a abordagem SOA nasceu da necessidade dos novos tipos de aplicação do fim do século XX, como “Cliente/Servidor, Internet, *batch*, *mobile* e *interenterprise EDP*”, que precisavam ir além das arquiteturas existentes até então. O Gartner esperava difundir a nova arquitetura substituindo as topologias tradicionais “que dificultam o compartilhamento de dados, e onde é quase impossível compartilhar lógica” ([Schulte e Natis, 1996](#)).

Os benefícios do SOA citados no material foram:

- “*Enables sharing of some business logic and data I/O logic among many applications of the same or different operating modes;*”
- *‘Automatically provides data sharing through the act of logic sharing;’*
- *‘Shields the developers of mode-specific and application-specific presentation, flow control and business logic from having to know how the back-end service functions are coded’*”.

SOA permitiu diferenciar tarefas cuja execução são iguais no servidor. O exemplo dado pelo Gartner é o de uma aplicação bancária e a tarefa é verificar o saldo em conta: todos os usuários têm acesso à essa tarefa e executam da mesma forma, mas o SOA permite que um usuário online tenha preferência frente a um *request* em *batch*, pois é lógico pensar que um usuário está esperando pela resposta enquanto uma requisição em *batch* não requer a mesma

urgência. O “serviço” é o responsável por checar essa prioridade e encaminhar os pedidos ao servidor, mas o passo a passo para buscar e devolver a informação é igual para ambos. *“It organizes software functions into modules in a way that maximizes sharing application code and data”* ([Schulte e Natis, 1996](#)).

Oito princípios práticos que definem SOA foram apresentados por Thomas Erl em meados dos anos 2000, e hoje são as principais regras a serem seguidas pelos interessados na arquitetura ([2004](#)):

1. Serviços compartilham um contrato formal: esse contrato é uma descrição do que o serviço faz e o que ele é capaz de entregar a partir das informações que recebe;
2. Serviços possuem baixo acoplamento: assegurar baixo acoplamento entre serviços permite que eles sejam mais independentes de detalhes de implementação e até de parâmetros estabelecidos no contrato;
3. Serviços são abstratos: esse princípio ajuda a definir o quanto serviços provedores abrem detalhes de seu funcionamento com consumidores de forma a permitir que ele próprio possa evoluir sem ferir o contrato estabelecido;
4. Serviços são reutilizáveis: serviços são pensados e desenhados para serem o mais reutilizáveis possível dentro de uma empresa, pois isso agiliza o processo de desenvolvimento;
5. Serviços são autônomos: serviços precisam ser capazes de governar a própria existência independente de outros serviços para diminuir riscos de baixa performance;
6. Serviços evitam informações de estados: pela sua capacidade de reutilização em outros contextos, é preciso tomar cuidado para que ao utilizar um serviço este não armazene informações demais específicas de uma aplicação correndo risco de baixa performance;
7. Serviços podem ser descobertos: serviços não devem estar escondidos de outros serviços pois o objetivo é que eles possam ser reutilizados, deve-se trabalhar para dar visibilidade aos seus contratos;

8. Serviços podem se compor: deve-se pensar em serviços como peças de uma composição maior que podem ser separados e reunidos de diferentes formas para entregar o valor desejado.

Para finalizar a explanação sobre SOA, vale enfatizar que esta não se trata de uma tecnologia e sim de uma lógica e conceitos arquiteturais, por isso não é preciso se limitar a ferramentas como *Web Services*, XML e BPM.

4.2 Desambiguação

A arquitetura de microsserviços nasceu de desafios reais encontrados nos dois principais modelos que a precederam: monolíticos e SOA, Richards (2015) pontua que essa estratégia de construção da arquitetura baseada em necessidades foi o que resultou no modelo ser tão difundido, pois ele não foi construído para depois se encontrar uma aplicação prática.

Nadareishvili et al. (2016) traz que o estilo arquitetural dos microsserviços foi criado a partir de padrões observados em algumas organizações pioneiras ao se desfazerem dos modelos monolíticos, e essas organizações não estavam conscientemente aplicando uma arquitetura estabelecida.

Dessa forma, é natural ver aspectos de várias práticas de outras arquiteturas na arquitetura de microsserviços. Por exemplo, dos oito princípios citados anteriormente, que podem ser seguidos de forma rígida ou não no SOA, a arquitetura de microsserviços traz um deles como absolutamente mandatório: independência total entre os serviços.

Richardson (2018) pontua três principais diferenças entre SOA e microsserviços:

1. A forma de trocar mensagens no SOA, com canais autorais com funcionalidades lógicas de mensagem e de negócio (os chamados “canais inteligentes”); no caso dos microsserviços as tecnologias de comunicação não requerem esse nível de complexidade e normalmente são *open-source*.
2. O tamanho dos serviços, onde o SOA basicamente comunica aplicações monolíticas entre si, os microsserviços não necessariamente são minúsculos, mas normalmente são pequenos e enquanto no ecossistema de SOA existem alguns serviços, no ecossistema de microsserviços existe dezenas a centenas deles.

3. No SOA é comum existir um *global data model* e bancos de dados compartilhados, enquanto nos microsserviços eles são completamente independentes.

Se antes era possível mapear exatamente onde estava cada banco de dados de cada aplicação, Langston (2018) traz uma das grandes mudanças sentidas na Amazon após a transição: “com microsserviços não é preciso se preocupar onde o serviço está rodando, não queremos nos preocupar com o servidor, nós nos preocupamos com o serviço e que ele está rodando de maneira adequada. E isso nos permitiu nos desfazermos do “*config management*” completamente”. O *config management* é o nome dado ao script de *deploy* utilizado para subir o código no servidor, e num sistema monolítico ele é o responsável por subir todas as dependências de uma vez, conforme o monolítico cresce, também cresce o *config management*.

Para fins deste trabalho, é utilizada a definição de Nadareishvili et al. (2015): “Um microsserviço é um componente independente de escopo limitado que oferece suporte à comunicação de forma transparente através de mensagens (se trata de um sistema interoperável). A arquitetura de microsserviços é um estilo de engenharia altamente automatizado e capaz de evoluir constantemente, composta de microsserviços orientados à resolver um problema específico”. Na figura 9 é possível visualizar um exemplo de aplicação web que utiliza microsserviços.

Richardson (2018) enumera os principais benefícios da adoção de microsserviços que deve ser analisado pelos interessados em adotar a arquitetura:

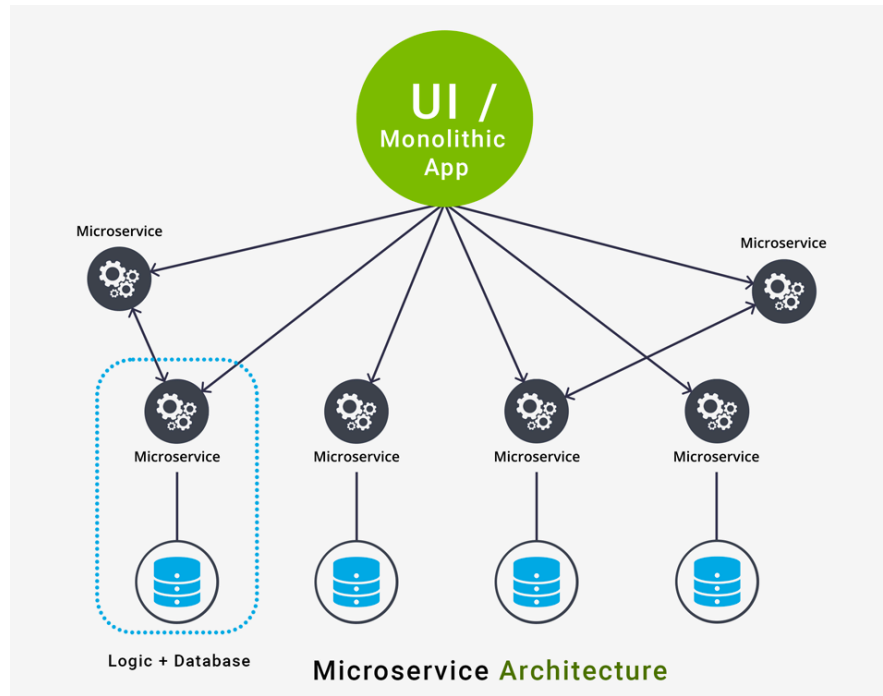
1. Permite a entrega e implantação contínuas de aplicações grandes e complexas;
2. Os serviços são pequenos e de fácil manutenção;
3. É possível fazer *deploy* de forma independente;
4. Os serviços são escaláveis independentemente;
5. A arquitetura de microsserviços permite que as equipes sejam autônomas;
6. Facilidade em experimentar e adotar novas tecnologias;
7. Melhor isolamento de falhas.

Em seguida, Richardson também traz os desafios da arquitetura:

1. Encontrar o conjunto certo de serviços é difícil;

2. Os sistemas distribuídos são complexos e, portanto, o desenvolvimento, o teste e a implantação são desafios;
3. A implantação de recursos que abrangem múltiplos serviços requer coordenação cuidadosa;
4. Decidir quando adotar a arquitetura de microserviços é difícil.

Figura 9: Representação da Arquitetura Microserviços



Fonte: *Microservices vs Monolithic Architecture*, Mayank Awasthi (2017)

Richards (2015) reitera: “um dos maiores desafios da arquitetura de microserviços está em determinar o nível de granularidade dos componentes de serviço”, pois quanto menos granular menos benefícios de *deploy*, escala, teste e desacoplamento terá. Já se o sistema for muito granular virão os desafios de orquestração de serviços sofridos pelo SOA.

4.4 Consequências Organizacionais da Adoção de Microserviços

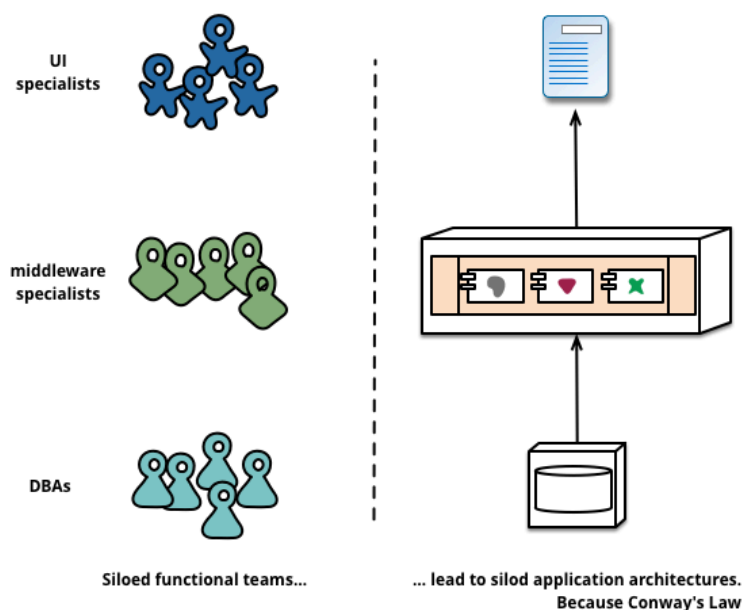
Existem duas características dos microserviços que causam impacto nos modelos monolíticos de desenvolvimento: autonomia e descentralização (Nadareishvili et al., 2016). Com microserviços não existirá uma pessoa ou uma área responsável na empresa por gerenciar ou controlar o que é adicionado, revisado ou retirado do sistema. O nível de

autonomia dos times que operam nessa arquitetura é o que traz a facilidade para modificar o serviço e a descentralização traz agilidade para lidar com problemas.

Esse nível de mudança organizacional só fará sentido se a empresa tiver ganhos reais com a arquitetura. Se a resposta é sim, com a mudança na área de desenvolvimento é necessário que a estrutura da organização evolua de acordo com a necessidade de comunicação, e essa necessidade vai depender do estágio em que a solução está. Conway (1968) completa: *“Because the design which occurs first is almost never the best possible, the prevailing system concept may need to change. Therefore, flexibility of organization is important to effective design”*.

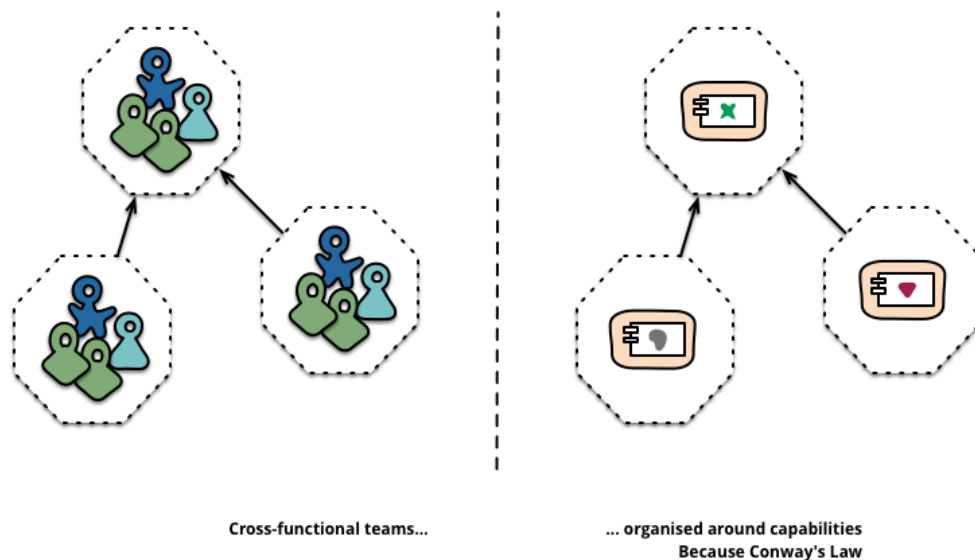
Na arquitetura de microsserviços, os serviços são divididos e limitados de acordo com o modelo de capacidade do negócio (*business capability*). O modelo de capacidade é utilizado para ilustrar as necessidades mais complexas dos negócios, a fim de criar soluções estratégicas que as atendam. Esses serviços exigem uma implementação de software de larga escala para essa área de negócios, incluindo *user interface* (UI), banco de dados e qualquer colaboração externa. Se um microsserviço precisa entregar uma capacidade de negócio, e essa capacidade só é entregue com uma implementação de larga escala, o time responsável pelo serviço será multidisciplinar. No modelo monolítico, ilustrado na figura 10, é possível ver que os especialistas separados por área.

Figura 10: Representação dos Times com Arquitetura Monolítica



Na arquitetura de microsserviços, os times são multidisciplinares para conseguir realizar uma entrega completa de capacidade do negócio. Podemos considerar que cada funcionalidade de um microsserviço possui uma expertise associada e o time será formado de acordo com as expertises que são necessárias e isso está ilustrado na figura 11.

Figura 11: Representação dos Times com Arquitetura Microsserviços



Fonte: *Microservices*, Martin Fowler ([2015](#))

Fowler ([2015](#)) complementa: “*Consequently the teams are cross-functional, including the full range of skills required for the development: user-experience, database, and project management*”.

Se uma empresa que adota o modelo da Figura 10 (com áreas especialistas em tecnologia, UI, servidor e banco de dados) precisa realizar uma mudança no sistema que afete mais de um desses times, essa mudança será multi-setorial, e mudanças que afetam mais de um time requerem aprovação de seus gerentes para alocação de tempo e recursos, o que gera um ônus operacional (que é feito para proteger organizações de hierarquia forte). Enquanto que numa organização que adota o modelo da Figura 11, um time que já possui todos os especialistas necessários será capaz de priorizar e entregar a solução, a mudança será muito mais rápida e envolverá muito menos pessoas.

Os benefícios dos microsserviços não são gratuitos. Assim como Richardson trouxe alguns aspectos negativos de sua adoção, ele também pontua o ônus do peso da arquitetura:

microsserviços utilizam ferramentas, automação e processos para o gerenciamento distribuído dos *status* do ecossistema. Por esse motivo, os custos de controlar e gerenciar aumentam significativamente, pois mesmo com microsserviços simples, a arquitetura se torna complexa. Dessa forma, antes de escolher realizar a transição da arquitetura atual para microsserviços, é preciso medir se os ganhos justificam o investimento ([Richardson, 2018](#)). Esse é o motivo pelo qual Fowler ([2015](#)) e Nadareishvili et al. ([2016](#)) acreditam que para começar um sistema, o modelo monolítico é o mais adequado, como explicado no capítulo 3.3.

5. Transição da Arquitetura Monolítica para Microserviços

Nesse capítulo, é sugerido um passo a passo para transição de arquitetura a partir de exemplos da Netflix (*streaming*), Amazon (Amazon Web Services), eBay, Google (Google App Engine), Wix e Gilt (varejo inglês online mencionado no livro de [Nadareishvili et al](#)).

Historicamente, as primeiras empresas a iniciar e finalizar o processo de transição de arquiteturas em produção foram as três primeiras acima citadas, mas as demais aprenderam e trouxeram boas práticas que facilitam a transição para empresas de menor porte, por esse motivo também foram adicionadas. Na tabela a seguir é possível ter uma visão cronológica das transições dessas empresas.

Tabela 1: Empresas e Responsáveis Técnicos que Realizaram a Transição de Sistemas Monolíticos para Microserviços

	Responsável	Ano da transição
Netflix	Josh Evans	2009
Amazon	Werner Vogels	2010
Ebay	Randy Shoup	2011
Google	Randy Shoup	2013
Wix	Aviran Mordo	2015
Gilt	Emerson Loureiro	2016

Fonte: Autor

5.1 Considerações Iniciais

Nadareishvili et al. (2016) traz três conceitos essenciais que devem ser levados em consideração pelas empresas que desejam adotar microserviços:

1. “São ideais para grandes sistemas: os problemas que a arquitetura de microserviços resolve são comuns à sistemas que já estão grandes e começaram a dificultar a escala do negócio”;

2. ‘São orientadas à objetivo: o foco é resolver os problemas que decorrem do sistema ter atingido um tamanho grande demais’;
3. ‘São focadas em substituição: a arquitetura facilita o processo de substituição ou remoção de componentes para atingir os objetivos. Manter componentes apenas porque eles sempre estiveram lá não faz parte da arquitetura de microsserviços’”.

É normal que algumas empresas já possuam parte dessas características: se utiliza SOA, então os conceitos de modularidade e comunicação baseada em mensagens já fazem parte do desenvolvimento, se implementa práticas de DevOps, então já investiu em *deploys* automatizados, e se utiliza metodologia ágil, então a cultura é preparada para receber microsserviços ([Nadareishvili et al., 2016](#)).

Existem alguns riscos associados a sistemas monolíticos que podem ser mitigados com a arquitetura de microsserviços. A dependência que funcionalidades têm entre si nos monolíticos pode tirar sistemas do ar completamente, enquanto com microsserviços partes dos sistemas são isoladas e aqueles serviços que são mais acessados podem ter um time adequado ao seu nível de vulnerabilidade; adequado em tamanho e nível de senioridade dos desenvolvedores.

Na Netflix, por exemplo, ficou mais fácil identificar e agir sobre ataques DDoS (*Distributed Denial of Service* ou Distribuição de Negação de Serviço) com microsserviços. O DDoS é uma forma de ataque hacker que consiste em sobrecarregar o servidor com requisições fantasma de acesso de forma que numa aplicação monolítica, por exemplo, o balanceador de carga não consiga suportar o volume de requisições e quebre antes que alguma ação seja tomada. Mas por utilizar microsserviços, a Netflix conseguiu visualizar o ataque e a autonomia dos times permitiu uma tomada de decisão rápida durante a ocorrência. Além disso, a independência dos serviços blindava a Netflix de sair do ar como um todo ([Evans, 2017](#)).

Outro desafio gerado pela dependência das funcionalidades é o peso operacional do *deploy*. Existem sistemas que levam algumas horas, outros que levam um dia inteiro, mas o pior na verdade é não saber exatamente quão demorado um *deploy* pode se tornar ([Langston, 2019](#)). Isso acontece porque independente de quão pequena é a alteração em uma

funcionalidade, todo o sistema está interligado, faz parte de um mesmo monolítico e precisa ser publicado junto.

Funcionalidades interligadas e *deploys* cada vez mais demorados geram monolíticos cada vez maiores, e com isso o desafio para a empresa em conhecer seu próprio *code base* como um todo. No WIX, esse foi o principal motivador para adotar a arquitetura de microsserviços: “Nós chegamos ao ponto de que ninguém conhecia o sistema por inteiro” ([Mordo, 2016](#)). E no SoundCloud, eles entenderam que a empresa possuía especialistas em partes do código o que prejudicava a entrega de valor ao cliente. “O código fonte que nós tínhamos era tão gigantes e tão amplo que ninguém conhecia ele todo. As pessoas desenvolveram suas áreas de expertise e custódia em submódulos da aplicação.” Phil Calçado, ex-Diretor de Engenharia do SoundCloud ([Nadareishvili et al., 2016](#)).

Dividir o sistema e deixá-lo sob a responsabilidade de times independentes traz outro benefício em termos de balanceamento de recursos de infraestrutura e pessoal. “*Some services require high availability, but are low in volume, and it’s the opposite for other services. A microservice approach allows us to tune for both of these situations, whereas in a monolith it’s all or nothing.*” Beier Cai, Diretor de Desenvolvimento, Hootsuite. ([Nadareishvili et al., 2017](#)). Evans ([2016](#)) complementa que, em algum momento, alguma parte do sistema irá falhar e aceitar isso significa desenvolver novas estratégias: para o Netflix, foi a “multi região”, onde os conteúdos foram replicados, duas vezes na América e uma na Europa, de forma que se algum dos provedores de conteúdos cai, existe outro (mais longe, porém disponível) que não deixará o cliente sem acesso.

Além dos aspectos técnicos, é preciso entender o impacto organizacional que a arquitetura monolítica carrega consigo para poder colher seus benefícios em totalidade. Como explicado no capítulo 3.2, o fluxo de comunicação de uma organização que entrega um sistema monolítico é fortemente hierarquizado. Com a mudança, decisões que antes eram centralizadas e passarão a ser responsabilidade de um ou mais times, isso impacta diretamente no corpo de líderes atuais da empresa. Os padrões (ferramentas, linguagem ou processo) não mais serão ditados de cima para baixo, eles se tornarão o padrão por serem a melhor alternativa dentre as demais ([Shoup, 2015](#)).

5.2 Desafios técnicos

A dependência de elementos externos compartilhados no sistema foi o que levou a Amazon a repensar completamente sua arquitetura, e a dar o primeiro passo para a transição. Vogels (2018) cita o pior dia de sua vida quando em 12 de dezembro de 2004, quando, por causa de um bug no banco de dados Oracle, a Amazon saiu completamente do ar por 12 horas no último dia de compras com entrega antes do Natal. A partir daí Vogels entendeu que a forma como o sistema estava projetado, abria espaço para vulnerabilidades externas e mesmo que se quisesse internalizar recursos, não seria suficiente replicar o modelo que era muito interdependente.

Sua ideia foi decompor o sistema em blocos menores, mais independentes, com o objetivo de diminuir o impacto das falhas, esses blocos são chamados de células. Vogels cunhou o termo “*blast radius*” (raio de explosão) para caracterizar o tamanho do impacto que um bug é capaz de causar em todo um sistema: “Diminuir o raio de explosão significa: se uma falha acontece, e lembre-se, tudo falha o tempo todo, seja conexão, hardware, transformadores ou o seu código, coisas falham e seu objetivo é minimizar o impacto que essa falha terá em seus clientes. Basicamente, se alguma coisa falha, o mínimo conjunto de clientes deverá ser afetado”.

É importante ressaltar que a Amazon não utiliza o termo microsserviços, na época sequer existia, e criou uma arquitetura própria chamada Arquitetura Baseada em Células, onde cada célula é independente uma da outra em termos de dados e lógica, e é capaz de escalar independentemente, também possui apenas um time responsável por ela, assim como em microsserviços. Mas a abstração da Amazon vai além, de duas formas. Primeiramente um conjunto de células faz parte de uma outra célula e existe hierarquia entre conjuntos de células. Além disso, uma célula é quebrada em unidades menores do que o escopo dos microsserviços, que se baseia em entrega em um objetivo de negócio. Porém, a transição da arquitetura na Amazon trouxe muito material que pode ser utilizado por empresas que desejam adotar a arquitetura de microsserviços.

O primeiro grande desafio técnico da transição é quebrar o sistema monolítico em serviços, já comentado no capítulo 2.2, pois para isso é necessário entender quais são as entregas completas do sistema que podem ser olhadas de forma independente. É importante

notar que se interrompida a transição para microsserviços, a organização pára com pedaços monolíticos que tendem a crescer (como qualquer sistema em produção que sempre possuirá bugs e melhorias), e isso aumentará ainda mais a complexidade da empresa que terá dois ou “n” grandes times cuidando de seus monolíticos.

No Wix, eles entenderam que as grandes entregas eram: visualizar e editar sites e, no primeiro momento, focaram em dividir o sistema em dois grandes monolíticos completamente independentes, mas ainda sim dois grandes monolíticos. Esse primeiro passo permitiu a quebra da equipe em dois grandes times que se especializaram em cada um dos dois sistemas, e não compartilhavam nem um banco de dados e eram capazes de fazer *deploys* independentes ([Mordo, 2016](#)). Sobre essa fase, Emerson Loureiro, Gilt, concorda ([2017](#)): “Faz parte do processo quebrar (o sistema) primeiro em serviços Java e depois em microsserviços”. Como visto no capítulo 4.1, serviços precisam ser completamente independentes, e a forma como o monolítico será quebrado dependerá da solução desenvolvida.

Sobre a independência dos times responsáveis pelos serviços, Evans ([2016](#)) complementa com um exemplo: “*If you have four teams working on a compiler you will end up with a four pass compiler*”. Ou seja, não existe um serviço compartilhado entre times. Um serviço bem definido é independente e requer apenas um time para sua manutenção.

A quebra do monolítico em serviços é um processo de aprendizagem que vai ensinar a organização quais os aspectos mais importantes na hora de extrair um serviço de outro serviço. Shoup ([2015](#)) argumenta que essa prática existe até hoje no Google: “Novos serviços são criados ou extraídos de serviços que já existem ou de aplicações que já existem, quando necessário, quando você precisa resolver um problema, então nós extraímos o serviço e colocamos um time por trás dele”. Os VPs do Google, eBay e Amazon que realizaram a transição fazem uma analogia recorrente dos microsserviços com organismos vivos: eles nascem de serviços preexistentes, eles vivem enquanto estão sendo utilizados e eles podem (e devem) ser descontinuados quando não são mais utilizados.

A estratégia utilizada no Gilt para substituir monolíticos foi baseada na teoria do Martin Fowler da “*StranglerFig*” (ou Figueira Estranguladora) ([2004](#)), onde um microsserviço é criado nos limites de um monolítico em produção, e vai extraindo partes do

monolítico para criar outros microsserviços até o monolítico ficar obsoleto. No Gilt ([Loureiro, 2016](#)), primeiramente era encontrado um “domínio de negócio” (uma completa entrega de valor) e o monolítico era quebrado ali em um novo monolítico. Em seguida para toda nova funcionalidade era criado um microsserviço que deveria interagir com o monolítico para entregar seu valor. A comunicação entre o novo microsserviço e o monolítico era feita a partir de uma camada temporária chamada: ACR - *Anti-Corruption Layer* (ou Camada Anti-Corrupção). A ACR é responsável por traduzir os tipos que o monolítico precisava dos tipos que o microsserviço enviava, e vice-versa, com o objetivo de proteger o novo microsserviço dos tipos do monolítico.

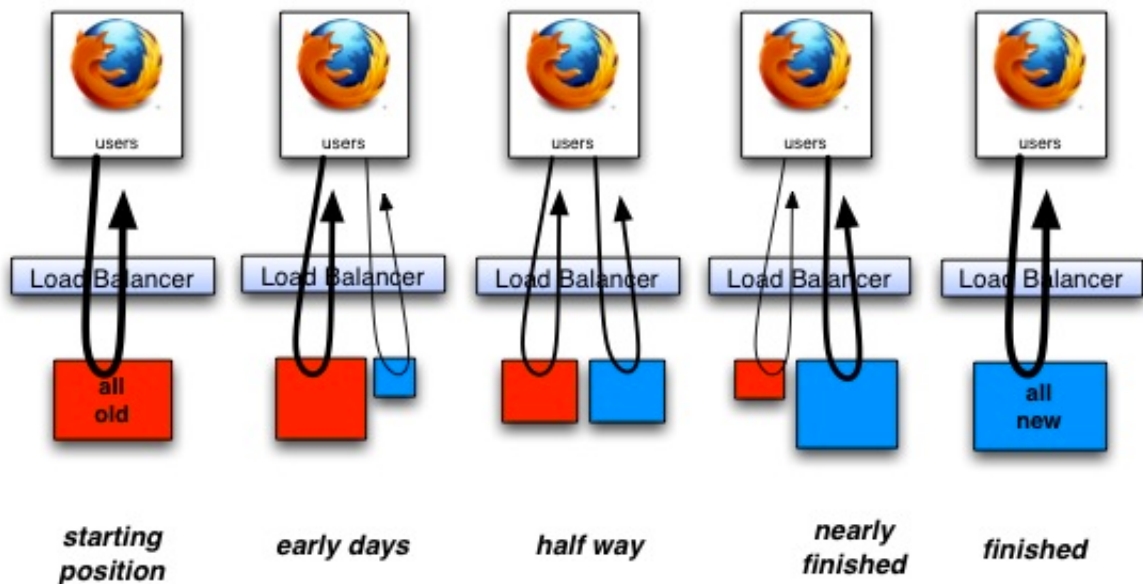
Com o novo microsserviço entregando a nova funcionalidade e o monolítico intacto, o time começou a retirar alguns tipos do monolítico para adicioná-lo a novos microsserviços adequados, na ordem dos tipos mais simples aos mais complexos, e o balanceador de carga era o responsável por encaminhar gradualmente as requisições do monolítico para o microsserviço até ser completamente seguro descontinuar o monolítico. Na figura 12 é possível ver um exemplo de aplicação web adicionando uma nova funcionalidade e descontinuando a antiga ([Hammant, 2013](#)). Sobre essa substituição, Loureiro traz algumas dicas do que fazer e não fazer:

- Faça: Adicione novos métodos, adicione parâmetros opcionais, *deprecate*, negocie mudanças muito disruptivas com outros times;
- Não faça: Evite mudanças disruptivas, não delete ou renomeie elementos, não adicione parâmetros obrigatórios, não remova parâmetros obrigatórios e não torne mudanças disruptivas obrigatórias.

Sobre o tamanho dos microsserviços, ele vai depender do tamanho do time que pode mantê-lo e o cálculo da quantidade de pessoas no time está na lei de Conway: a capacidade de comunicação da organização vai ditar o design do sistema ([Mordo, 2016](#) e [Conway, 1968](#)). No capítulo 5.3, a formação dos times é abordada em profundidade.

Para estabelecer um limite na hora de quebrar microsserviços, Mordo ([2016](#)) complementa trazendo uma comparação prática: um microsserviço tem uma função, um objetivo operacional, uma entrega completa de valor, enquanto uma biblioteca, por exemplo, tem uma função apenas computacional.

Figura 12: Migração de Funcionalidade Utilizando o Método da Figueira Estranguladora



Fonte: *Legacy Applications*, Paul Hammant (2013)

Evans (2017) contribui com outro aspecto da implementação do microsserviço, ele defende a automatização de processos de rotina com o objetivo de desonerar o time de atividades repetitivas para que ele possa abraçar um novo serviço. Na Netflix, isso foi alcançado criando um ciclo de aprendizado que termina na automação e adoção da prática por todos os serviços. As etapas do processo são: Incidente - Resolução - Revisão - Remediação - Análise - Melhor prática - Automação - Adoção. *“This is how knowledge becomes code and gets integrated in all your microservices architecture”*.

Sobre ferramentas, Mordo (2016) traz uma importante consideração para os responsáveis pela transição: é muito atraente querer utilizar várias ferramentas de uma vez, mas é preciso ponderar o nível de complexidade que cada uma delas traz antes de tornar uma regra. Algumas ferramentas não serão necessárias desde o primeiro dia, pois ainda haverá alguns monolíticos minimamente conectados ao sistema. *“Everytime you introduce another middleware to your system, you're introducing DevOps overhead. Remember: you're running on a production system and the most important thing with a production system is that it keeps working”*. Quanto a algumas ferramentas para manter o controle dos microsserviços, como

Zookeeper e Eureka, Mordo afirma que no Wix eles só passaram a utilizar essas ferramentas ao atingir 200 microsserviços.

5.3 Desafios organizacionais

Para falar dos desafios organizacionais, este item inicia a partir dos times por trás dos microsserviços. No Gilt, os times têm entre três e cinco pessoas que são independentes de outros times para nutrir seu microsserviço: requisitos, qualidade, *deploy*, código fonte e tecnologia, tudo é escolha do time responsável.

Os sistemas monolíticos passam a sensação de segurança para a organização, pois as mudanças obedecem a uma hierarquia, e a responsabilidade está sempre no nível acima. Mas em um ecossistema de microsserviços isso não existe. Os times possuem um grau de responsabilidade maior, e a área de desenvolvimento se torna uma comunidade de *service owners* (donos de serviços). Cada dono de serviço assume dois papéis diariamente: o de provedor de serviço e o de consumidor de serviço. No Gilt, Loureiro traz alguns princípios importantes para cada um desses papéis ([2016](#)):

Princípios do Consumidor de Serviços:

- *Design for failure*: quando projetar seu microsserviço, leve em consideração que todo sistema quebra, e que aquele serviço que você quer acessar pode não estar disponível em algum momento;
- *Plan to be throttled*: se um serviço está tendo problemas de performance, pode ser que esse problema seja desencadeado para os serviços consumidores, e é papel do consumidor não quebrar por isso;
- *Plan to retry (with exponential backoff)*: se seu serviço quebrar, tenha uma rotina para reiniciar de forma inteligente e não quebrar novamente;
- *Degrade gracefully*: se algum serviço do qual o seu depende não está respondendo de maneira adequada, não o bombardeie de mensagens de erro, por exemplo;
- *Cache when appropriate*: uma ótima forma de tirar a pressão de outros serviços que você depende é armazenar algumas informações em sua cache, como dados de nome de um usuário por exemplo: não mudam com tanta frequência;

Princípios do Provedor de Serviços:

- *Publish standard metrics*: todos os times devem entrar em acordo sobre quais são as métricas necessárias para que se possa utilizar um serviço, e é muito importante que todos possam ver como está a saúde dos serviços em um único lugar;
- *Publish logs for aggregation*: todos os serviços devem manter *logs* de seu funcionamento para fácil visualização;
- *Be prepared to throttle*: é importante se preparar para o caso do seu serviço ter problemas de performance, e é preciso ter em mãos mecanismos para agir rápido sobre esse problema;
- *Implementation details are private*: manter detalhes de implementação privados é uma forma de não criar dependência de outros serviços nesses detalhes, e assim garantir a independência de implementação de todos;
- *Maintain backwards compatibility*: quando for preciso realizar mudanças em seus serviços, tenha mecanismos para informar futuras depreciações, e dê tempo para que os outros serviços se atualizem de suas mudanças;

A hierarquia de uma organização que trabalha com a arquitetura de microsserviços tende a ser mais horizontal. Um impacto interessante que a transição deixou no Google foi a extinção do cargo de “arquiteto de software”, segundo Shoup (2015). Como as decisões não seriam mais centralizadas e *top-down* (pois não existia mais uma pessoa ou um time de pessoas que pudessem responder por toda a área de App Engine do Google) esses engenheiros de software poderiam ser distribuídos nos times e auxiliar no desenvolvimento. Lembrando que o papel da arquitetura de software é garantir que os objetivos do negócio são cumpridos, e esse papel passou a ser dos próprios times.

É válido evocar o novo paradigma que os microsserviços trouxeram para o Google: “*Every service at Google is either deprecated or not ready yet*” (Shoup, 2015). Os serviços passam por revisões para entender se ele ainda é necessário, e pode ser extinguido a qualquer momento. O time responsável por esse serviço já possui outros na carteira, e agora poderá priorizar serviços que são mais úteis à outras partes da empresa. “*Services justify their*

continued existence through being used and once the service is no longer being used we'd serve deprecated and retire it and the team can go on and do something else". Não existe um número mágico, mas dentre as empresas analisadas, nenhuma delas relacionava os serviços 1x1 aos times.

5.4 Percepções finais

Em 12 de dezembro de 2004, Werner Vogels viveu seu pior dia na Amazon quando um bug no banco de dados deixou o site fora do ar por 12 horas no último dia de compras com entrega antes do Natal. A falta de controle de seus recursos para escalar (o banco de dados era Oracle), fez com que a Amazon criasse sua própria infraestrutura para o e-commerce, e, mais tarde, comercializasse o que hoje conhecemos como Amazon Web Services, “o primeiro modelo de banco de dados nativo da nuvem” ([Vogels, 2018](#)), uma plataforma capaz de escalar conforme requisitado pelo usuário e aberta à inovação.

A transição do Netflix para a arquitetura de microsserviços se deu no momento de mudança de modelo de negócio (DVDs para *streaming*), e a pergunta que Evans queria responder naquele momento era: qual é a arquitetura certa para o longo prazo? A resposta que ele obteve de um engenheiro sênior da época foi: “você se importa com os impactos organizacionais?” ([2016](#)). A transição do Netflix durou pouco mais de um ano, e hoje a empresa é uma das maiores provedoras de conteúdo técnico (estudo e ferramentas) para empresas que também querem seguir esse caminho.

O eBay e o Google (Google App Engine) tiveram o mesmo engenheiro responsável pelas transições, Randy Shoup; e hoje ele é um dos maiores defensores do modelo de “Arquitetura Evolutiva”, pois segundo ele: “*There is no perfect architecture for all products and all scales*”.

O Wix passou pela transição em 2015, e trouxe muitas provocações sobre o que não fazer e porque não se preocupar com as últimas ferramentas disponíveis, com o alerta da criação do *overhead* para o DevOps. O Gilt conseguiu casar conceitos teóricos de Martin Fowler no seu processo de migração que permitiu trazê-los como exemplo com o adicional das consequências reais de implementação, tanto positiva quanto negativa. A seguir, alguns

benefícios citados pelo VP de Engenharia, Adrian Trenaman após a transição ([Nadareishvili et al., 2016](#)):

- “Menos dependência entre times, resultando em ganho de produção;
- ‘Várias iniciativas puderam acontecer em paralelo;
- ‘Foi possível utilizar múltiplas tecnologias, linguagens e frameworks;
- ‘Permitiu uma graciosa degradação de serviços;
- ‘Hoje é mais fácil falhar e seguir em frente: possibilidade de inovar com ‘códigos descartáveis’”.

Tabela 2: Sumário de Passo a Passo para Transição para Microsserviços

Etapa	Atividade	Desafio
1	Entender quais são os gargalos de desenvolvimentos na solução atual e se é possível mitigá-los com microsserviços.	Ter clareza dos desafios técnicos intrínsecos à utilização de microsserviços dentro do escopo da solução atual.
2	Convencimento da organização da necessidade de migração para microsserviços.	As mudanças organizacionais decorrentes de microsserviços impactam na cultura e na forma como a empresa se comunica.
3	Quebrar o monolítico em novos monolíticos menores, é a primeira fase de divisão de times multi-disciplinares e autônomos.	Identificar dentro de um monolítico quais as entregas de valor que podem ser quebradas em monolíticos menores. Redistribuir lideranças entre os times para manutenção dos serviços.

Etapa	Atividade	Desafio
4	Acrescentar correções e novas funcionalidades utilizando a metodologia da Figueira Estranguladora. As etapas 3 e 4 são recorrentes e devem acabar apenas quando o monolítico estiver pronto para ser completamente descontinuado.	Não permitir que partes do monolítico infectem os microsserviços, recomenda-se utilizar uma camada de segurança intermediária. Implantar a cultura de times donos de serviços: hora provedores, hora consumidores.
5	Identificar dentro de microsserviços se ele não cresceu o suficiente para: ou ser quebrado em mais de um ou para ter parte dele descontinuada. Etapa recorrente da arquitetura de microsserviços.	Os times provedores de serviços precisam estar abertos à abrir mão de serviços que já não estejam entregando valor para a solução.

Fonte: Autor

6. Conclusão

Cada vez mais empresas tem escolhido abandonar sua estrutura monolítica para adotar microsserviços, uma arquitetura que promete agilizar entregas sem abrir mão de segurança. Não existe uma arquitetura “bala de prata”, que resolve todos os problemas de todas as empresas de tecnologia, mas existem aquelas que melhor se adequam ao momento e objetivos de cada empresa, trazendo diversas consequências em sua adoção. Os impactos de adotar microsserviços vão além da nova modularização do sistema e alcançam a forma como a organização se comunica, por isso é preciso mais do que a aprovação dos arquitetos de software para utilizar esse modelo.

A busca por novas arquiteturas capazes de permitir a entrega de softwares de forma rápida e confiável vem desde os anos 60, com as primeiras modularizações de Dijkstra e Parnas. Essa busca levou a criação de vários padrões ao longo dos anos como arquitetura em camadas, baseada em eventos, microkernel e microsserviços. As três primeiras tentaram resolver problemas de forma inédita (oferecendo primeiro a solução para que o projetista encaixasse seus problemas), enquanto a arquitetura de microsserviço mais herdou características de seus predecessores do que inventou, resolvendo problemas que os projetistas já sofriam.

Mesmo assim, pela facilidade de desenvolvimento e acesso a ferramentas, o modelo monolítico é amplamente difundido como a melhor alternativa para iniciar implementações de sistemas, mesmo aqueles que pretendem escalar. Mas os desafios encontrados por empresas que crescem e escalam são vários: desconhecimento da base de código completa, *deploys* demorados e imprevisíveis, dificuldade de adicionar novas funcionalidades e as ações para resolução de bugs e quebras são demoradas, por envolverem mais de uma área de desenvolvimento. A jornada de monolítico para microsserviços é o caminho defendido por Fowler, Nadareishvili e Richards como o mais benéfico.

Por ser uma arquitetura que herdou muitas características de outras, os defensores de microsserviços alertam para que complexidade desnecessárias não entrem no projeto de forma a evitar repetir erros de seus predecessores. Os principais motivos que atraem lideranças de tecnologia são: a capacidade de manter um ritmo de entregas contínuas de pequenas e grandes

funcionalidades – serviços são pequenos e de fácil manutenção, são independentes o que permite *deploys* mais rápidos –, escalabilidade e isolamento de falhas, além de ter times autônomos que são capazes de agir sobre bugs, falhas e quebras rapidamente e com facilidade de experimentar novas tecnologias. Ter em mãos os problemas que microsserviços resolvem permite a uma liderança de tecnologia mapear hoje quais são os desafios de sua empresa que podem ser resolvidos com microsserviços.

As vantagens de usar microsserviços são vastas e a complexidade que a arquitetura carrega não são escondidas, o processo de transição vai modificar a forma como a organização pensa em desenvolvimento e entregas e é preciso estar aberto a reinventar o modelo de comunicação organizacional. Alguns desafios encontrados são: definição dos serviços, complexidade inerente a sistemas distribuídos e a coordenação entre serviços (relação provedor e consumidor).

A diversidade de sistemas e modelos de negócio do mercado levanta a dúvida para os casos em que a adoção de microsserviços não seja a melhor opção. Este trabalho se limitou a casos de empresas que oferecem seus serviços online em escala global e como os benefícios dos microsserviços se encaixam nesse modelo de negócios. Para contribuir com o estudo das limitações da arquitetura de microsserviços, é interessante analisar casos de insucesso de migração: seja pelo modelo de negócio limitado (de baixa escala), a complexidade de implantação ou pelas barreiras culturais que não permitiram as mudanças organizacionais.

7. Referências

- [1] NADAREISHVILI, MITRA, MCLARTY, AMUNDSEN. (2016). *“Microservice Architecture”*. Editora: O’reilly. v 1 p 9 - 17.
- [2] NADAREISHVILI, Irakli. (14 de Abril de 2017). *“Changing Banking for Good, One Microservice at a Time”*. Disponível em: <<https://medium.com/capital-one-developers/rule-of-twos-and-microservice-architecture-3f57db7f6896>> Acesso em: 11 de setembro de 2018.
- [3] CONWAY, Melfin E. (1968). *“How do Committess Invent”*. Editora: F. D. Thompson Publications, Inc. v 1 p 4.
- [4] CONWAY, Melfin E. (2010). *“Committees Paper”*. Disponível em: <http://www.melconway.com/Home/Committees_Paper.html>. Acesso em: 19 de Abril de 2019.
- [5] FOSTER, Richard N. (1986), *“Innovation: The Attacker's Advantage”*. Editora: Summit Books. p 6
- [6] FOSTER, Richard N. (2001). *“Creative Destruction: Why Companies That Are Built to Last Underperform the Market--And How to Successfully Transform Them”*. Editora: Crown Business. p 12
- [7] RICHARDSON, Chris. (2018). *“Microservice Patterns MEAP”*. Editora: Manning. v 11 p 4 - 26.
- [8] RICHARDSON, Chris. (2018). *Pattern: Monolithic Architecture*. Disponível em: <<https://microservices.io/patterns/monolithic.html>>. Acesso em: 19 de Abril de 2019.
- [9] BASS, CLEMENTS, KAZMAN. (2013). *Software Architecture in Practice*. Editora: Pearson Education, Inc. v 1 p 17. [LINK](#)
- [10] RICHARDS, Mark. (2015). *“Software Architecture Patterns”* O’Reilly v1 p 5 - 15.
- [11] RITO, Anaury N. P. (2017). A Economia das APIs e Microserviços. Trabalho de Conclusão de Curso em Ciência da Computação - UFPE, Recife.
- [12] AWASTHI, Mayank. (2017). *“Microservice vs Monolithic Architecture”*. Disponível em: <<https://nodexperts.com/blog/microservice-vs-monolithic/>>. Acesso em: 19 de Abril de 2019.
- [13] ROBBINS, Stephen P. (2013). *Organizational Behavior*. Editora: Prentice Hall. v 15 p 480.

- [14] SHOUP, Randy. (2015). *Service Architectures at Scale: Lessons from Google and eBay*. Disponível em: <<https://www.infoq.com/presentations/service-arch-scale-google-eBay>> Acesso em: 20 de Abril de 2019.
- [15] SHOUP, Randy. (2014). *Evolutionary Architecture - Good Enough is Good Enough*. Disponível em <<http://www.randyshoup.com/evolutionary-architecture>> Acesso em: 19 de Maio de 2019.
- [16] SCHULTE, W. Roy, NATIS, Yefim. (1996). ““Service Oriented’ Architectures, Part I”” Disponível em: <<https://www.gartner.com/en/documents/302868>> Acesso em: 19 de Maio de 2019.
- [17] FOWLER, Martin. (2005). “*MonolithFirst*”. Disponível em: <<https://martinfowler.com/bliki/MonolithFirst.html>>. Acesso em 19 de Abril de 2019.
- [18] FOWLER, Martin. (2004). “*StranglerFigApplication*”. Disponível em: <<https://martinfowler.com/bliki/StranglerFigApplication.html>> Acesso em 25 de Maio de 2019.
- [19] FOWLER, Martin. (2015). “*Break Monolith Into Microservices*”. Disponível em: <<https://martinfowler.com/articles/break-monolith-into-microservices.html>> Acesso em 19 de Abril de 2019.
- [20] FOWLER, Martin. (25 de março de 2014). “*Microservices, A Definition of this New Architectural Term*”. Disponível em: <<https://martinfowler.com/articles/microservices.html>> Acesso em: 12 de setembro de 2018.
- [21] ERL, Thomas. (2008). “*Principles of Service Design*”. Editora Pearson Education. v1 p10.
- [22] LANGSTON, Brant. (2018). “*AWS re:Invent 2018: From Monolith to Microservices (And All the Bumps along the Way) (CON360-R1)*” <https://www.youtube.com/watch?v=gahR0_ZrYHs> Acesso em: 19 de Maio de 2019.
- [23] MORDO, Aviran. (2016). “GOTO 2016 • Journey from Monolith to Microservices & DevOps • Aviran Mordo” <<https://www.youtube.com/watch?v=jeMNaKIrsu8>> Acesso em: 19 de Abril de 2019.
- [24] VOGELS, Wernel. (2018). “*AWS re:Invent 2018 - Keynote with Werner Vogels*”<<https://www.youtube.com/watch?v=femopq3JWJg>> Acesso em: 25 de Maio de 2019.

- [25] LOUREIRO Emerson. (2016). “AWS re:Invent 2016: From Monolithic to Microservices: Evolving Architecture Patterns (ARC305)” <<https://www.youtube.com/watch?v=oRIYtOsAlzk>>. Acesso em: 19 de Abril de 2019
- [26] EVANS, Josh. (2016). “*Mastering Chaos - A Netflix Guide to Microservices*” <<https://www.youtube.com/watch?v=CZ3wIuvmHeM>> Acesso em: 19 de Abril de 2019.
- [27] HAMMANT, Paul. (2013). *Legacy Application Strangulation : Case Studies* Disponível em: <<https://paulhammant.com/2013/07/14/legacy-application-strangulation-case-studies/>>. Acesso em 25 de Maio de 2019.
- [28] HOROWITZ, Andreessen. (01 de setembro de 2016). “*a16z Podcast: All about Microservices with Adrian Cockcroft, Frank Chen, and Martin Casado*”. Disponível em: <<https://a16z.com/2016/09/01/microservices/>> Acesso em: 09 de setembro de 2018.
- [29] INDRASIRI, Kasun. (28 de agosto de 2017). “*Pragmatic Microservices*”. Disponível em: <<https://medium.com/microservices-in-practice/microservices-in-practice-7a3e85b6624c>> Acesso em: 12 de setembro de 2018.
- [30] WILLMOTT, Steven. (2014). “*What are some challenges faced when building APIs to expose and integrate systems of large enterprises?*” Disponível em: <<https://www.quora.com/What-are-some-challenges-faced-when-building-APIs-to-expose-and-integrate-systems-of-large-enterprises>>. Acesso em: 16 dez. 2018
- [31] HENSHALL, Adam. (2017). “*How 4 Top Startups are Reinventing Organizational Structure*”. Disponível em: <<https://www.process.st/organizational-structure/>>. Acesso em: 19 de Abril de 2019.
- [32] CLARK, Simon. (1980). “*What is Fordism*” Disponível em: <<https://homepages.warwick.ac.uk/~syrbe/pubs/Fordism.pdf>>. Acesso em: 19 de Abril de 2019.
- [33] RAYMOND, Eric S. (2003). *The Art of Unix Programming*. Disponível em: <<https://nakamotoinstitute.org/static/docs/taoup.pdf>> Acesso em: 19 de Abril de 2019.